

Q1 REPORT

YUSUF ERAY ERİŞ

1. Rank and the Number of Vertices

The starting question of this project is the following:

If the rank of the adjacency matrix is known, what is the maximum number of vertices of a twin-free graph?

A graph is called **twin-free** if no two vertices have the same set of neighbors. In terms of the adjacency matrix, each vertex corresponds to one row of the matrix. Therefore, if two vertices have the same neighbors, then the corresponding two rows of the adjacency matrix are equal.

So the twin-free condition means that all rows of the adjacency matrix must be different.

The rank of a matrix can be understood as the number of independent rows. If the adjacency matrix has rank r , then all rows are contained in an r -dimensional row space. A first rough expectation is that an r -dimensional binary space can contain up to

2^r

different binary vectors. Therefore, one may first expect that a twin-free graph with adjacency matrix rank r could have at most around 2^r vertices.

However, the adjacency matrix of an undirected graph is symmetric. This means that if one vertex is connected to another vertex, then the reverse connection must also exist. In matrix form,

$$A_{ij} = A_{ji}$$

So we cannot choose the rows completely independently. When we connect one vertex to another vertex, the other vertex is also connected back to the first one. In other words, one edge affects two symmetric positions in the adjacency matrix.

Because of this symmetry restriction, the naive 2^r expectation is too large. A more realistic comparison is around

$2^{r/2}$

The purpose of this project is to investigate the repeated double cover construction and compare the number of vertices of the constructed graphs with the rank of their adjacency matrices.

2. The Graph Construction

We start with the graph

$$\Gamma_0 = (\{x\}, \emptyset)$$

which has only one vertex and no edge.

Given a graph

$$\Gamma = (V, E)$$

the double cover graph Γ' is constructed as follows:

The vertices of Γ' consist of two copies of V , together with one additional vertex v .

The extra vertex v is adjacent to all vertices in only one copy of V , but it is not adjacent to the vertices in the other copy.

Then the repeated construction is defined by

$$\Gamma_1 = \Gamma_0'$$

and

$$\Gamma_{i+1} = \Gamma_i'$$

for $i = 1, 2, 3, \dots$

For each graph Γ_i , I investigated the following properties:

$$V(\Gamma_i)$$

$$\text{rank}(A_i)$$

and

$$\text{Aut}(\Gamma_i)$$

Here, A_i is the adjacency matrix of Γ_i

3. Finding the Matrix Transition Algorithm

Before writing the code, I first tried to understand how the adjacency matrix changes from one depth to the next.

Assume that the adjacency matrix at depth n is

A_n .

If A_n is an $m \times m$ matrix, then the next graph has two copies of the old vertex set plus one new vertex. Therefore, the next matrix has size

$(2m+1) \times (2m+1)$.

So the vertex count changes as

m to $2m+1$.

Starting from one vertex, this gives

$$|V(\Gamma_n)| = 2^{n+1} - 1.$$

the next step was to decide how to place the old matrix inside the new matrix.

The old matrix A_n is copied into the corner blocks of the new matrix. Then the new vertex v is added to the outside as the last row and the last column.

The block structure is:

$A_n \quad 0 \quad 1$

$0 \quad A_n \quad 0$

$1 \quad 0 \quad 0$

Here:

A_n

is the adjacency matrix from the previous depth.

1

means that the new extra vertex is connected to every vertex in one copy.

0

means that the new extra vertex is not connected to the other copy.

So the algorithm is:

First, create a new zero matrix of size

$(2m+1) * (2m+1)$.

Then copy the previous matrix A_n into the correct corner blocks.

After that, add the new vertex v as the last row and last column.

Finally, connect this new vertex to all vertices in only one copy of the previous vertex set.

4. Implementation in Orbiter

I implemented the construction by following Orbiter's existing graph creation structure.

First, I added the new command information in `create_graph_description`. In this part, I defined a flag for the command and a variable for the depth value. This allows Orbiter to store whether `-yusuf_eray_double_cover` is used and which depth is requested.

```
f_yusuf_eray_double_cover = false ;  
double_cover_depth = 0;
```

Then I updated the command reading part. When Orbiter reads `-yusuf_eray_double_cover` depth from the terminal, it activates the flag and stores the given depth value.

```
else if(ST.stringcmp(argv[i], "-yusuf_eray_double_cover") == 0){  
    f_yusuf_eray_double_cover = true;  
    double_cover_depth = ST.strtoi(argv[++i]);  
    if(f_v){  
        cout<< "-yusuf_eray_double_cover "<< double_cover_depth << endl;  
    }  
}
```

Also i added print function.

```
if(f_yusuf_eray_double_cover){  
    cout << "-yusuf_eray_double_cover " << double_cover_depth << endl;  
}
```

After that, I moved to the `create_graph` part. Here, I added a new condition for my command. If the command flag is active, Orbiter calls the function that creates the repeated double cover graph.

The actual graph object is created in a separate function. This function sends the depth value to the graph theory domain and sets labels for the graph, so the output files can be recognized easily.

```
else if(description->f_yusuf_era_double_cover){
    if(f_v){
        cout<< "create_graph::init "<< "before create_yusuf_era_double_cover
"<<endl;
    }
    create_yusuf_era_double_cover(
        description->double_cover_depth,
        verbose_level);
    if(f_v){
        cout<< "create_graph::init "<< "after create_yusuf_era_double_cover
"<<endl;
    }
}

void create_graph::create_yusuf_era_double_cover(
    int depth,int verbose_level){
    int f_v = (verbose_level >= 1) ;
    if (f_v) {
        cout << "create_graph::create_yusuf_era_double_cover" << endl;
    }

    combinatorics::graph_theory::graph_theory_domain GT;

    if (f_v) {
        cout << "create_graph::create_yusuf_era_double_cover "
                "before GT.make_yusuf_era_double_cover" << endl;
    }

    GT.make_yusuf_era_double_cover(Adj,N,depth,verbose_level);

    if (f_v) {
        cout << "create_graph::create_yusuf_era_double_cover "
                "after GT.make_yusuf_era_double_cover" << endl;
    }

    label = "yusuf_era_double_cover_" + std::to_string(depth);
    label_tex = "{\\rm yusuf\\_era\\_double\\_cover\\_"
        + std::to_string(depth) + "}";
    if (f_v) {
        cout << "create_graph::create_yusuf_era_double_cover done " << endl;
    }
}
```

The main matrix construction is done in `graph_theory_domain`. The function starts from the one-vertex graph with adjacency matrix `[0]`. Then, for each depth, it creates a new matrix of size $2N + 1$, copies the previous adjacency matrix into the correct blocks, and adds the new extra vertex as the last row and column. This new vertex is connected to only one copy of the previous graph.

```
void graph_theory_domain::make_yusuf_eray_double_cover(
    int* & Adj, int &N,
    int depth, int verbose_level){

    int f_v = (verbose_level >= 1);

    if (f_v) {
        cout << "graph_theory_domain::make_yusuf_eray_double_cover" << endl;
    }
    if (depth < 0) {
        cout << "graph_theory_domain::make_yusuf_eray_double_cover depth < 0" <<
endl;
        exit(1);
    }

    N = 1 ;
    Adj = NEW_int(N * N);
    Adj[0] = 0;
    int * temp;
    int last_n = N;
    int first_rectangle_index;

    for (int i = 0; i<depth; i++){
        last_n = N;
        N = (N * 2) + 1;
        temp = NEW_int(N * N);
        Int_vec_zero(temp, N * N);
        for(int j = 0; j < last_n ; j++){
            for(int k = 0; k < last_n ; k++){
                first_rectangle_index = k + (N * j);
                temp[first_rectangle_index] = Adj[k + (j *last_n)];
                temp[first_rectangle_index + (last_n *N)+last_n] = Adj[k + (j
*last_n)];
            }
            temp[(N-1) + (N * j)] = 1;
            temp[j + ((N - 1) * N)] = 1;
        }

        FREE_int(Adj);
        Adj = temp;
    }
    if (f_v) {
        cout << "graph_theory_domain::make_yusuf_eray_double_cover done" << endl;
    }
}
```

Finally, I added the necessary declarations to the header files so that the new variables and functions can be used across the Orbiter source files.

In summary, the command is read in `create_graph_description`, passed to `create_graph`, and the adjacency matrix is constructed in `graph_theory_domain`.

5. Testing and Computation

After implementing the command, I tested it using Orbiter graph theoretic activities.

The main activities used were:

-properties

-eigenvalues (to compute and report eigenvalues)

-eigenvalue_report

-automorphism_group (to calculate automorphism group order)

-export_csv (to export the adjacency matrix)

The rank was obtained from the eigenvalues. Since the adjacency matrix of an undirected graph is symmetric, its rank over the real numbers is the number of nonzero eigenvalues.

So,

$\text{Rank}(\text{adj}) = N - \text{zeroeigenvaluesamount}$

Note : Due to C++ floating-point precision limits, values around $e-17$ should be mathematically treated as exactly zero.

6. Results

The computed values are summarized in the following table.

	Γ_0	Γ_1	Γ_2	Γ_3	Γ_4	Γ_5	Γ_6	Γ_7
Rank of adj	0	2	6	14	30	62	126	254
Automorphism group order	1	2	4	16	256	65536	4294967296	18446744073709551616
Number of vertices	1	3	7	15	31	63	127	255
Number of zero eigenvalues	1	1	1	1	1	1	1	1

9. Interpretation of the Results

The original motivation was to compare the number of vertices of a twin-free graph with the rank of its adjacency matrix.

A first simple expectation was that if the adjacency matrix has rank r , then the number of different rows could be as large as

$$2^r.$$

However, since the adjacency matrix is symmetric, one cannot choose the rows independently. When one vertex is connected to another vertex, the other vertex is also connected back. Therefore, a more realistic comparison is around

$$2^{r/2}$$

The repeated double cover construction does not come close to this value.

In the computed examples, the rank is almost equal to the number of vertices:

$$\text{rank}(A_i) = |V(\Gamma_i)| - 1.$$

So if

$$r = \text{rank}(A_i),$$

then

$$|V(\Gamma_i)| = r + 1.$$

This is very far from $2^{r/2}$

Therefore, this construction appears to waste rank. Each time the graph grows, the rank also grows almost by the same amount. This means that the construction does not produce many vertices for a fixed rank.

In other words, although the number of vertices grows quickly with the depth, the rank also grows almost equally quickly. Because of this, the construction is not efficient for producing large twin-free graphs with small adjacency matrix rank.

10. Conclusion

In this project, I implemented the repeated double cover graph construction as a new Orbiter command.

The command

`-yusuf_eray_double_cover depth`

constructs the graph Γ_i for the given depth i .

The implementation was done by adding the command description, terminal parsing, graph creation skeleton, graph theory domain construction, and the required header file declarations.

The matrix construction was found by analyzing how the adjacency matrix changes from depth n to depth $n+1$. The old matrix is copied into the new block matrix, and the new vertex is added as the last row and last column. This new vertex is connected to all vertices in only one copy of the previous graph.

The computations show that

$$|V(\Gamma_i)| = 2^{i+1} - 1.$$

The ranks computed from the eigenvalues show the pattern

$$\text{Rank}(A_i) = |V(\Gamma_i)| - 1.$$

Therefore, this graph family does not seem to be close to the expected exponential behavior such as

$$2^{r/2}$$

Instead, the number of vertices is approximately linear in the rank:

$$|V(\Gamma_i)| = r + 1.$$

So the repeated double cover construction gives a clean and structured family of graphs, but it appears to waste rank and does not produce many vertices for a fixed adjacency matrix rank.

11. References

Anton Betten, **Orbiter User's Guide**.

Anton Betten, **Graph Theory with Orbiter**.

Anton Betten, **Orbiter source code**.

ChatGPT, **AI-based assistance tools were used for debugging, command syntax checking, and report organization.**